

Universidad Nacional Autónoma de México

Facultad de Ciencias

Simulación de algoritmos de planificación de CPU

Algoritmo Round Robin y Shortest Job First

22 de septiembre de 2026

1. Introducción

La planificación de procesos es una de las funciones fundamentales de un sistema operativo. Cuando existen varios procesos que requieren utilizar el procesador el sistema operativo debe determinar cuál de ellos recibirá el recurso y durante cuánto tiempo.

Los algoritmos de planificación de CPU establecen diferentes estrategias para organizar el acceso de los procesos al procesador. Entre los algoritmos existentes se encuentra Round Robin, este algoritmo es especialmente utilizado en sistemas donde se busca proporcionar una distribución equitativa del tiempo de CPU entre los procesos.

Otro algoritmo que estaremos viendo es Shortest Job First, se utiliza principalmente cuando se conoce el tiempo estimado de todos los procesos lo cual es bastante sencillo de implementar.

En este caso implementamos el algoritmo *Round Robin*, para analizar su comportamiento realizamos simulaciones utilizando diferentes valores de *quantum*, en particular utilizamos $q = 2$, $q = 4$ y $q = 6$. Además de implementar por medio de un arreglo de procesos el algoritmo Shortest Job First, utilizando un listado pequeño de procesos y uno más grande.

2. Algoritmos implementados

2.1. Round Robin

Round Robin es un algoritmo de planificación, donde dentro del sistema operativo se asigna a cada proceso una porción de tiempo equitativo y ordenada, donde tratamos a los procesos con la misma prioridad, este fue diseñado especialmente para sistemas de uso compartido.

Vamos a definir un intervalo de tiempo al que le llamaremos quantum, cuya duración va a variar dependiendo del sistema, y la cola de procesos la vamos a estructurar como una cola circular. El planificador la va a recorrer asignando un quantum de tiempo a cada proceso, la organización de nuestra cola va a ser FIFO.

Los procesos listos para ejecutarse se mantienen en una cola. Y el proceso que se encuentra al frente de la cola recibe la CPU durante un máximo de q unidades de tiempo.

Si el proceso termina durante este intervalo, se elimina de la cola. En cambio, si todavía necesita tiempo de CPU, es colocado nuevamente al final de la cola.

El funcionamiento general lo podemos resumir de la siguiente manera:

1. Se agregan a la cola los procesos que ya han llegado.
2. Se selecciona el proceso que se encuentra al frente de la cola.
3. El proceso se ejecuta durante un máximo de q unidades de tiempo.
4. Si el proceso termina, se registran sus métricas.
5. Si el proceso no termina, se coloca nuevamente al final de la cola.
6. Se agregan los procesos que hayan llegado durante la ejecución.
7. El procedimiento continúa hasta que todos los procesos hayan terminado.

El objetivo de utilizar diferentes valores de quantum es observar cómo cambia el comportamiento del algoritmo al modificar la duración del intervalo de ejecución.

2.2. Shortest Job First

El algoritmo asigna a la CPU al proceso que tenga una ráfaga de ejecución más corta de los procesos listados conocidos con su tiempo de llegada, teniendo como objetivo el acortar el tiempo de espera promedio. Usualmente se implementa en su variante no apropiativa, donde un proceso retiene la CPU hasta terminar pero puede provocar el clásico problema del que el proceso con mayor tiempo nunca se ejecute mientras sigan llegando procesos de menor tamaño. Existe otra variante apropiativa conocida como Shrotest Remaining Time First, que interrumpe un proceso en ejecución si llega uno nuevo con tiempo menor.

Su funcionamiento general se puede definir como:

1. Se inicializa el tiempo del reloj del sistema en 0.
2. Se identifican los procesos disponibles y se filtra únicamente aquellos cuyo tiempo de llegada sea menor o igual al tiempo actual del sistema.
3. Si ningún proceso cumple con la condición de llegada, entonces se incrementa el reloj del sistema hasta que llegue el próximo proceso.
4. Se busca el proceso con el menor tiempo de ejecución del listado de procesos disponibles, en este caso, si dos o más procesos tienen la misma ráfaga de ejecución se usa el tiempo de llegada como desempate.
5. Se ejecuta el proceso seleccionado, y se actualiza el tiempo actual del sistema sumando la cantidad de tiempo de dicho proceso.
6. Se registra el tiempo como completado y se elimina de la cola, para volver al paso 2 hasta que todos los procesos hayan finalizado.

3. Estructuras de datos utilizadas

Para la implementación utilizamos principalmente dos estructuras: una estructura para representar los procesos y una cola circular para administrar los procesos listos.

3.1. Estructura Process

Cada proceso se representa mediante una estructura que almacena información relacionada con su ejecución.

Los principales campos utilizados son:

- **id**: identificador del proceso.
- **arrival**: tiempo en el que llega el proceso.
- **burst**: cantidad de tiempo de CPU que necesita.
- **priority**: prioridad asignada al proceso.
- **remaining**: tiempo de CPU que todavía necesita el proceso.
- **start_time**: primer instante en el que el proceso recibe la CPU.
- **finish_time**: instante en el que termina.

- `waiting_time`: tiempo total que permaneció esperando.
- `turnaround`: tiempo transcurrido desde su llegada hasta su finalización.
- `response_time`: tiempo transcurrido desde su llegada hasta la primera vez que recibe la CPU.
- `is_completed`: es una variable para indicar si el proceso ya se completo, se utiliza únicamente en la implementación de **SJF**.

3.2. Cola circular

Para implementar Round Robin se utilizamos una cola circular.

La cola almacena los índices de los procesos que se encuentran listos para utilizar la CPU.

Se utilizaron tres variables principales:

- `frente`: indica la posición del proceso que será atendido.
- `final`: indica la posición donde se insertará el siguiente proceso.
- `cantidad`: indica cuántos procesos se encuentran actualmente en la cola.

La característica circular se consigue mediante la operación:

$$\text{posición siguiente} = (\text{posición actual} + 1) \bmod MAX_PROCESOS$$

Esto permite reutilizar las posiciones iniciales del arreglo cuando se alcanza el final.

3.3. Arreglo

Para la implementación de **SJF**, un arreglo estático se utiliza principalmente como un pool de los procesos conocidos y disponibles para su ejecución, permitiendo escoger de manera dinámica el proceso más óptimo en cada instante de tiempo de la computadora, utilizando un ciclo `for` para iterar sobre los elementos y en este caso, al marcar el atributo `is_completed` como `true`, no es necesario sacarlo del arreglo. Para futuras simulaciones, se puede implementar una lista ligada para eliminar procesos finalizados y agregar nuevos procesos disponibles.

4. Implementación

El programa fue implementado en lenguaje C. Cada proceso se representa mediante cuatro datos:

$$ID, \quad arrival, \quad burst, \quad priority$$

Por ejemplo:

1	1	0	2	1
2	2	2	4	2
3	3	4	6	3

La prioridad se almacena dentro de la estructura del proceso debido a que forma parte del formato de entrada (en este caso). Sin embargo, en la simulación de Round Robin esta información no interviene en la selección del siguiente proceso, ya que la prioridad es la misma para todos; y en el caso de **SJF** la prioridad es por medio del valor **burst**.

4.1. Reinicio de las simulaciones

Debido a que el programa ejecuta Round Robin varias veces utilizando el mismo conjunto de procesos, antes de cada simulación tenemos que reiniciar las métricas.

En particular, se van a restablecer:

- tiempo restante;
- tiempo de inicio;
- tiempo de finalización;
- tiempo de espera;
- tiempo de retorno;
- tiempo de respuesta.

Esto nos va a permitir que cada ejecución con un quantum diferente sea independiente de las ejecuciones anteriores.

Mientras que con la simulación de **SJF**, se debe de reiniciar el tiempo de reloj de la computadora a 0 y el listado de procesos disponibles.

5. Métricas utilizadas

Para analizar el comportamiento del algoritmo se utilizamos tres métricas principales.

5.1. Tiempo de retorno

El tiempo de retorno (*turnaround time*) corresponde al tiempo transcurrido desde que el proceso llega hasta que termina:

$$T_{\text{retorno}} = T_{\text{finalización}} - T_{\text{llegada}}$$

5.2. Tiempo de espera

El tiempo de espera representa el tiempo durante el cual el proceso permaneció esperando para utilizar la CPU.

Se calcula mediante:

$$T_{\text{espera}} = T_{\text{retorno}} - T_{\text{burst}}$$

donde T_{burst} representa el tiempo total que el proceso efectivamente utiliza la CPU.

5.3. Tiempo de respuesta

El tiempo de respuesta corresponde al intervalo entre la llegada del proceso y la primera vez que recibe la CPU:

$$T_{\text{respuesta}} = T_{\text{inicio}} - T_{\text{llegada}}$$

Esta métrica es especialmente importante en sistemas interactivos, pues nos permite observar cuánto tiempo debe esperar un proceso antes de comenzar a recibir atención del procesador.

6. Datos de entrada

Para realizar las pruebas se utilizó un conjunto de 100 procesos.

Los procesos se almacenaron en un archivo de texto y cada línea contiene cuatro valores:

```
1 ID  Arrival  Burst  Priority
```

Los tiempos de llegada avanzan en intervalos de dos unidades y los tiempos de CPU utilizados por los procesos siguen el patrón:

2, 4, 6, 8, 10

La prioridad también sigue un patrón de valores entre 1 y 5. (pero en este caso RR no lo utiliza para decidir que proceso pasa primero).

Un ejemplo de las primeras líneas del archivo es:

```
1 1 0 2 1
2 2 2 4 2
3 3 4 6 3
4 4 6 8 4
5 5 8 10 5
6 6 10 2 1
```

El archivo completo contiene 100 procesos.

7. Compilación y ejecución

Para compilar el proyecto, se utiliza la herramienta GNU Make junto con el compilador GCC con soporte para el estándar C11. El comando empleado en la raíz del proyecto es:

```
1 make
```

Este proceso utiliza las banderas `-Wall`, `-Wextra` y `-Wpedantic` para la validación del código durante su compilación y construcción. Después de la compilación, se generan los ejecutables `roundRobin_simulador` y `sjf_simulador`, los cuales estarán ubicados en la carpeta `build/`. Si se desea eliminar los archivos y ejecutables generados durante la compilación, se debe utilizar:

```
1 make clean
```

En cuanto a la ejecución, ambos simuladores comparten la misma sintaxis para procesar los datos de entrada y generarán sus respectivos reportes automáticamente dentro de la carpeta **resultados/**. Se pueden ejecutar pasando el archivo de texto como parámetro o leyendo directamente desde la entrada estándar.

8. Prueba de funcionamiento

El programa para Round Robin realiza tres simulaciones consecutivas utilizando el mismo conjunto de 100 procesos.

Las simulaciones corresponden a:

1. Round Robin con $q = 2$.
2. Round Robin con $q = 4$.
3. Round Robin con $q = 6$.

En cada ejecución se muestran los intervalos en los cuales cada proceso utiliza la CPU. Por ejemplo, la salida comienza mostrando información de la forma:

1	Tiempo	0 - 2	: P1
2	Tiempo	2 - 6	: P2
3	Tiempo	6 - 10	: P3
4	...		

Posteriormente se muestran los resultados individuales de cada proceso y finalmente los promedios.

Para el programa de **SFJ**, se manda únicamente el archivo con los 100 procesos, mostrando los resultados individuales de cada uno y los tiempos promedios.

9. Análisis de resultados

9.1. Resultados para Round Robin

A partir de los resultados obtenidos se puede observar que el cambio del quantum modifica de manera importante las métricas de planificación.

9.1.1. Tiempo de espera

El tiempo de espera promedio obtenido fue:

$$250.16 \rightarrow 240.64 \rightarrow 230.86$$

al pasar de $q = 2$ a $q = 4$ y posteriormente a $q = 6$.

Para el conjunto de procesos utilizado en esta práctica, el promedio disminuyó conforme aumentó el quantum.

9.1.2. Tiempo de retorno

El comportamiento del tiempo de retorno promedio fue:

$$256.16 \rightarrow 246.64 \rightarrow 236.86$$

para $q = 2$, $q = 4$ y $q = 6$, respectivamente.

Al igual que el tiempo de espera, el tiempo de retorno promedio disminuyó en las simulaciones realizadas al aumentar el quantum.

9.1.3. Tiempo de respuesta

En contraste con las métricas anteriores, el tiempo de respuesta promedio aumentó:

$$71.52 \rightarrow 122.82 \rightarrow 157.12$$

al incrementar el quantum.

Esto muestra que, para el conjunto de datos utilizado, el cambio del quantum produce un comportamiento diferente dependiendo de la métrica que estemos analizando.

9.1.4. Comparación de los diferentes quantums

La Tabla 1 permite comparar directamente las tres ejecuciones.

Cuadro 1: Comparación de los resultados obtenidos.

Quantum	Espera	Retorno	Respuesta
2	250.16	256.16	71.52
4	240.64	246.64	122.82
6	230.86	236.86	157.12

Los resultados nos muestran que no todas las métricas presentan el mismo comportamiento ante el cambio del quantum.

Para este conjunto específico de procesos, los tiempos promedio de espera y retorno disminuyeron al aumentar el quantum, mientras que el tiempo de respuesta promedio aumentó.

9.2. Resultados para SFJ

A partir del resultado obtenido, podemos tener la siguiente información de las métricas de planificación tras simular los 100 procesos bajo el enfoque no apropiativo:

9.2.1. Tiempo de espera

El tiempo de espera promedio de la simulación fue de 129.78.

9.2.2. Tiempo de retorno

El tiempo de retorno promedio general registrado para los procesos fue de 135.78

9.2.3. Tiempo de respuesta

El tiempo de respuesta promedio fue de 129.78. Al tratarse de una variante no apropiativa del algoritmo SJF, el tiempo de respuesta promedio es exactamente igual al tiempo de espera promedio, ya que los procesos no son interrumpidos una vez que se les asigna la CPU por primera vez.

9.3. Comparación de resultados de Round Robin y SFJ

Cuadro 2: Comparación general de métricas promedio entre Round Robin y SJF.

Algoritmo / Configuración	Espera	Retorno	Respuesta
Round Robin ($q = 2$)	250.16	256.16	71.52
Round Robin ($q = 4$)	240.64	246.64	122.82
Round Robin ($q = 6$)	230.86	236.86	157.12
SJF (No Apropiativo)	129.78	135.78	129.78

Shortest Job First (SJF) es más eficiente a la minimización del tiempo de espera y del tiempo de retorno de las iteraciones de Round Robin. El tiempo de espera promedio obtenido con SJF (129.78) lo cual es aproximadamente la mitad de lo logrado por Round Robin (230.86 con $q = 6$).

En contraste, Round Robin supera a SJF en la interactividad del sistema cuando se configura con quantums pequeños. Con $q = 2$ (71.52) y $q = 4$ (122.82), Round Robin logra tiempos de respuesta inferiores a los de SJF (129.78). Debido a que la implementación de SJF analizada es no apropiativa, la CPU puede ser monopolizada temporalmente por procesos largos que llegaron primero, retrasando la primera atención de los demás. Round Robin, por diseño, fuerza la rotación de la CPU, sacrificando tiempos de retorno globales para garantizar que todos los procesos sean atendidos rápidamente tras su llegada.

10. Consideraciones de implementación

Durante la implementación se tomaron algunas decisiones importantes.

10.1. Uso de una cola circular

La utilización de una cola circular permite implementar de manera natural la política de Round Robin, ya que los procesos que no terminan después de consumir su quantum regresan al final de la cola.

La operación circular nos evita tener que desplazar todos los elementos del arreglo cada vez que extraemos un proceso.

10.2. Procesos que llegan durante la ejecución

Después de ejecutar cada proceso se revisa si existen nuevos procesos cuya llegada sea menor o igual al tiempo actual.

Estos procesos son agregados a la cola de procesos listos.

Por lo tanto, si un proceso nuevo llega exactamente al finalizar un quantum, se incorpora a la cola antes de volver a insertar el proceso que acaba de ser ejecutado.

11. Limitaciones

La simulación utiliza un arreglo con una capacidad máxima de:

$$MAX_PROCESOS = 200$$

Por lo tanto, el programa puede almacenar hasta 200 procesos.

Además, el algoritmo Round Robin utilizado en esta práctica no considera la prioridad para decidir qué proceso debe ejecutarse. Aunque la prioridad se encuentra almacenada en la estructura `Process`, esta información no interviene en la selección de procesos de Round Robin.

12. Conclusiones

La implementación nos permitió ejecutar los algoritmos Round Robin y Shortest Job First (SJF) para obtener métricas clave relacionadas con el desempeño de los procesos.

Para el conjunto de 100 procesos utilizado, los resultados de Round Robin mostraron que al aumentar el quantum de 2 a 4 y posteriormente a 6, el tiempo de espera promedio pasó de 250.16 a 240.64 y finalmente a 230.86. El tiempo de retorno promedio presentó un comportamiento similar, pasando de 256.16 a 246.64 y posteriormente a 236.86. Por otra parte, el tiempo de respuesta promedio aumentó de 71.52 con un quantum de 2, a 122.82 con un quantum de 4, y a 157.12 con un quantum de 6.

En contraste, la simulación de SJF bajo el mismo conjunto de datos logró minimizar sustancialmente los promedios globales, registrando un tiempo de espera y de respuesta de 129.78, junto con un tiempo de retorno de 135.78.

Estos resultados nos permiten observar que modificar el quantum afecta de manera diferente a las distintas métricas. Por lo tanto, el comportamiento de Round Robin depende tanto del valor del quantum como de las características del conjunto de procesos que se está ejecutando. Asimismo, la integración de SJF evidencia algunos contrastes teóricos: mientras SJF es óptimo para reducir el tiempo de espera de un lote de procesos, Round Robin sacrifica la eficiencia para evitar que un proceso monopolice el sistema.

Dado que en general Round Robin teóricamente tiene un tiempo de espera relativamente grande pero nos garantiza un reparto justo de la CPU arrojando tiempos de respuesta buenos para la interactividad, y SJF demuestra una alta eficiencia despachando procesos rápidos, podemos decir que nuestros resultados son satisfactorios y consistentes con la naturaleza de ambos algoritmos.

13. Referencias

Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*.
- [2] Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*.

- [3] Wikipedia. *Planificación Round-robin*. Wikipedia, la enciclopedia libre. Disponible en: https://es.wikipedia.org/wiki/Planificaci%C3%B3n_Round-robin [Consultado: 20 de septiembre de 2026].
- [4] Wikipedia. *Shortest job next*. Wikipedia, la enciclopedia libre. Disponible en: https://en.wikipedia.org/wiki/Shortest_job_next [Consultado: 20 de septiembre de 2026].